ECE 213 Assignment 2

Part A

Name: Abhinandan Sharma

PID: A69041851

GPU Model: NVIDIA A30

Part B

Fixing maxPairs = 5000

Fixing blockNum = 400

After parallelizing the entire code:

Time taken by alignmentOnGPU kernal on optimized code (with shared memory access) = 405472926 ns

Time taken by alignmentOnGPU kernel with global memory access = 456007410 ns (total time 1759 ms)

Speedup due to shared memory over global memory = 1.125

Part C

Fixing T = 200

Fixing maxPairs = 5000

Fixing blockNum = 400

After parallelizing the entire code:

Time taken by alignmentOnGPU kernal on optimized code (with wavefront parallelism) = 405472926 ns

Time taken by alignmentOnGPU kernal without wavefront parallelism = 8829848602 ns

Speedup due to wavefront parallelism = 21.777

Explanation:

The 22x speedup comes from wavefront (anti-diagonal) parallelism. In the DP scoring matrix, all cells on diagonal k satisfy i + j = k and depend only on diagonals k-1 and k-2, making them fully independent of each other. So, instead of thread 0 computing them one by one, all 256 threads split the diagonal with {i = i_start + tx; i += blockDim.x}, computing up to 200 (because T=200) cells simultaneously, with a "__syncthreads()" barrier between each diagonal to enforce the data dependency. The speedup is around 22x rather than the theoretical ~200x maximum because short corner diagonals leave most threads idle, and the per-diagonal barrier adds overhead.

Part D

The maximum value of T I was able to set is **200**, constrained by the 48KB shared memory limit per block, with the dominant consumer being tbDir[T*T], which grows quadratically (40,000 bytes at T=200). Tiling is far more suitable for hardware accelerators than full matrix or banded alignment because it keeps the working set small enough to fit entirely in on-chip memory (shared memory on GPUs, BRAM on FPGAs), avoiding the repeated costly DRAM accesses. Tiling is an efficient way of increasing the arithmetic intensity by maximizing the reuse of data which shift the roofline plot from memory bound region to compute bound region, and extra cores of parallel hardware can then help improve the performance. Each tile also exposes a natural unit of wavefront parallelism up to T independent cells per diagonal, mapping directly onto the SIMD thread model of GPUs and the pipelined systolic architecture of FPGAs in a way that full matrix and banded methods simply cannot match.

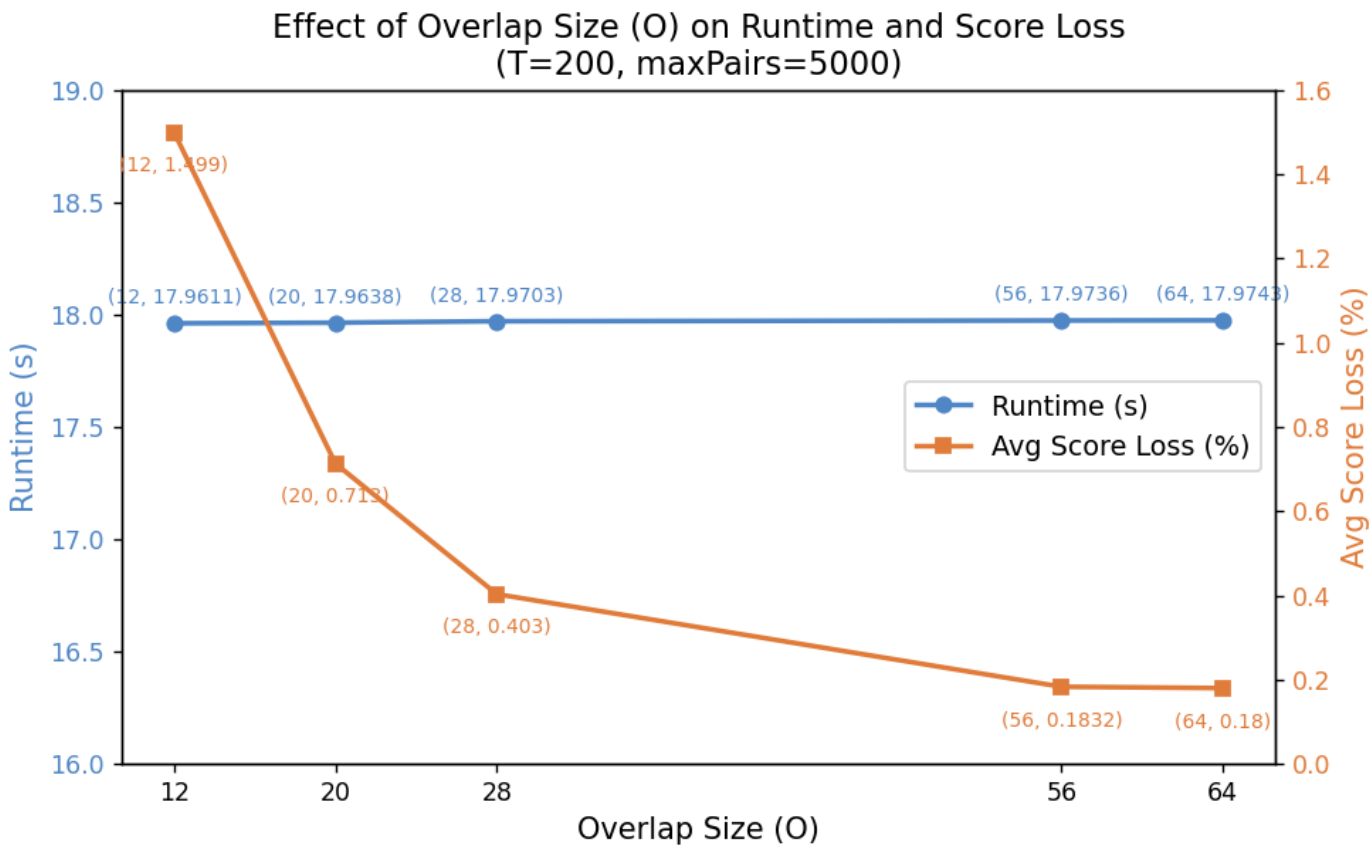
Part E

Fixing maxPairs = 5000

Fixing batchSize = 400

Fixing T = 200

Fixing 0 = [12, 20, 28, 56, 64]



As overlap size O increases from 12 to 64, score loss decrease monotonically and runtime remains pretty much constant (slight increasing trend). The score loss drops sharply from 1.499% to 0.18% because a wider overlap region gives the GACT heuristic a better chance of finding the true optimal exit point before advancing to the next tile, reducing path drift. The flat runtime with decreasing score loss is expected behaviour for this setup. With T=200 and 5000 pairs, the GPU is fully saturated and all 5000 blocks are scheduled simultaneously and the SMs are already running at maximum occupancy regardless of O. Changing O does not change the number of blocks, threads, or memory transfers, so the GPU scheduler sees an identical workload every time and wall-clock runtime stays constant.

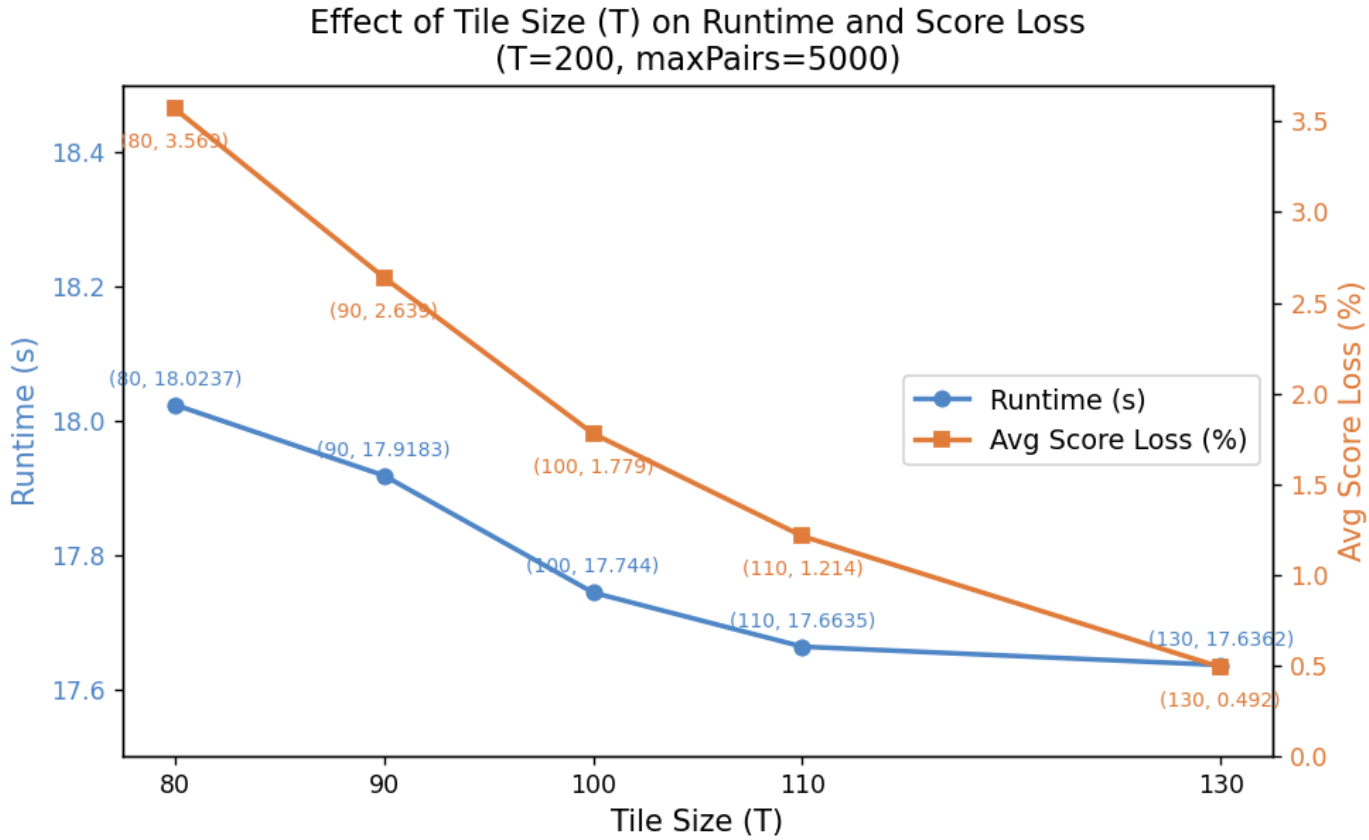
Part F

Fixing maxPairs = 5000

Fixing batchSize = 2

Fixing O = 64

Fixing T = [80, 90, 100, 110, 130]



With a fixed O, increasing T reduces the total number of tiles needed to cover the full sequence, since each tile covers more ground. Fewer tiles means less total overhead (fewer barrier syncs, traceback passes, and shared memory resets), so runtime decreases. Score loss also decreases because a larger tile computes a more complete DP matrix before the overlap heuristic picks the exit point, reducing drift between tiles. Both metrics improve together because they share the same root cause — larger T simply means fewer, higher-quality tiles.

Part G

numThreads = 4

batchSize = 1000

numBlocks = 1000

blockSize = 256

T = 200

O = 64

Total program time = 1684 ms

Average Percentage Score Loss = 0.18%

```
144     const int T = 200;          // Tile size
145     const int O = 64;           // Overlap between tiles
146
147     const int BLOCK_SIZE = 256;
148
149     // Scoring Scheme (DO NOT MODIFY)
150     const int16_t MATCH = 2;
151     const int16_t MISMATCH = -1;
152     const int16_t GAP = -2;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
Consolidate compiler generated dependencies of target aligner
[ 25%] Built target compare_alignment
[ 87%] Built target check_alignment
[100%] Built target aligner
Setting CPU threads to 4 and printing GPU device properties.
Device Number: 0
  Device name: NVIDIA A30
  Device memory: 25231032320
  Memory Clock Rate (KHz): 1215000
  Memory Bus Width (bits): 3072
  Peak Memory Bandwidth (GB/s): 933.120000

Program completed in 1684 ms

1.07user 0.62system 0:01.80elapsed 93%CPU (0avgtext+0avgdata 406496maxresident)k
0inputs+193656outputs (0major+55901minor)pagefaults 0swaps
[SUCCESS] All 10000 aligned sequences match their raw originals.

Valid Comparisons: 5000 pairs. Average Percentage Score Loss: 0.180053%

pod "abs021-4081608" deleted
abs021@codeserver:~$
```

Part H

Fixing maxPairs = 5000

Fixing batchSize = 2

```
Generated:
    /home/abs021/assgn2-abhinandansharma1999/build/report106.nsys-rep
    /home/abs021/assgn2-abhinandansharma1999/build/report106.sqlite
========= COMPUTE-SANITIZER
Setting CPU threads to 4 and printing GPU device properties.
Device Number: 0
  Device name: NVIDIA A30
  Device memory: 25231032320
  Memory Clock Rate (KHz): 1215000
  Memory Bus Width (bits): 3072
  Peak Memory Bandwidth (GB/s): 933.120000

Program completed in 12687 ms

========= ERROR SUMMARY: 0 errors
[SUCCESS] All 10000 aligned sequences match their raw originals.

Valid Comparisons: 5000 pairs. Average Percentage Score Loss: 0.180053%

pod "abs021-3824544" deleted
abs021@codeserver:~$
```

Program time without compute-sanitizer = 1864 ms

Program time with compute-sanitizer = 23477 ms

Part I

Yes, it is possible to observe negative score loss. The reference was generated using banded alignment with a fixed band width of 500, which restricts the DP computation to a diagonal corridor and will miss the optimal alignment path if it deviates beyond that band, even with 98% average sequence identity, the ~2% divergent regions can cause the true optimal path to briefly exit the band, forcing suboptimal gap placements. Our GACT implementation computes a full (T×T) DP matrix within each tile with no band restriction, so it can find better local paths that banded alignment missed. To maximize the chance of achieving a higher score than the reference, we would set T as large as the shared memory limit allows (T=200) and O close to T (e.g. O=180), so that each tile covers a large region with a conservative, well-anchored exit point that minimizes drift between tiles, effectively approximating a full alignment over each window and making it strictly more powerful than a fixed band when the optimal path deviates.

Bonus

a)

To replace the max comparison logic with DPX instructions, I used Hopper's integer dynamic programming intrinsic __viaddmax_s16, which performs an add-and-max operation in a single instruction. Instead of sequential if comparisons, I computed the maximum between (score_diag, score_up) and then between that result and score_left using DPX intrinsics. A compile-time architecture guard (__CUDA_ARCH__ >= 900) ensures that DPX instructions are only used on Hopper GPUs, while earlier architectures (e.g., A30) fall back to standard max() operations. The traceback direction is determined after the DPX max computation by comparing the final score against the candidate values, preserving correctness. This approach keeps the algorithm identical functionally but leverages hardware-accelerated DP operations when available.

src > alignment.cu

```cpp
else {
    // -- Inner Matrix Calculation --
    // TODO: Replace this global memory access with shared memory
    char r_char = shared_ref[i - 1];
    char q_char = shared_qry[j - 1];

    int16_t score_diag = wf_scores[prepre_k + (i - 1)] + (r_char == q_char ? MATCH : MISMATCH);
    int16_t score_up   = wf_scores[pre_k + (i - 1)] + GAP;
    int16_t score_left = wf_scores[pre_k + i] + GAP;

    // Find Max (BONUS: Replace with DPX instructions)
    #if __CUDA_ARCH__ >= 900
        // DPX accelerated max3
        int best = __vimax3_s16x2(
            (score_diag & 0xFFFF) | (score_diag << 16),
            (score_up   & 0xFFFF) | (score_up   << 16),
            (score_left & 0xFFFF) | (score_left << 16)
        );

        score = (int16_t)(best & 0xFFFF);
    #else
        score = score_diag;
        if (score_up > score)   score = score_up;
        if (score_left > score) score = score_left;
    #endif

    // Direction (still needed for traceback)
    if (score == score_diag)      direction = DIR_DIAG;
    else if (score == score_up)   direction = DIR_UP;
    else                           direction = DIR_LEFT;
}
```

b)

Performing traceback on the GPU and transferring only the compact direction path to the CPU is generally a good design choice. The DP matrix is large and expensive to transfer, while the traceback path is relatively small (O(n) instead of O(n^2)). Reconstructing aligned strings on the CPU is lightweight string manipulation and does not benefit significantly from GPU parallelism. Performing full traceback on the CPU would require transferring the entire DP matrix, which would incur substantial memory bandwidth overhead. Therefore, computing the DP and traceback on the GPU while reconstructing final strings on the CPU provides a good balance between computation efficiency and data transfer cost.